Programming II — Assignment 1

Peer Review Report

Music Streaming Platform · IMC UAS Krems · 2026

Preamble — Purpose and Instructions for Reviewers

This peer review is a formative exercise. Its purpose is to provide your fellow student with constructive, evidence-based feedback on the current state of their implementation — even if their work is incomplete. You are not grading your peer; rather, you are helping them identify strengths to build upon and concrete areas for improvement before the final submission deadline.

Your review will be read by both the recipient and the course lecturer. The quality and fairness of your feedback will itself be assessed and will contribute to 5% of your own course grade. You are therefore accountable for producing a thorough, honest, and respectful report.

What you ARE expected to do:

- Check out (clone) the reviewed student's repository at a specific point in time and record that date.
- Work through each section of this report systematically, providing both a numerical score (1–5) and a written commentary.
- Refer to specific locations in the code when making observations (e.g., file name, class name, method name, or approximate line number).
- Run the test suite locally (pytest) and report what you observe.
- Be specific and constructive — vague praise or vague criticism is of no value to your peer.

What you are NOT expected to do:

- Fix bugs or rewrite any part of the reviewed code.
- Evaluate design decisions that were prescribed by the assignment (the class hierarchy and diagram were given — do not critique those choices).
- Make assumptions about intent; evaluate only what is actually present in the repository.

Tone:

Every critical observation must be phrased constructively and referenced to a specific piece of code or requirement. A comment such as "the code is messy" is not acceptable. A comment such as "the method record_session() in platform.py contains deeply nested conditional blocks that reduce readability — consider extracting the inner logic into a helper method" is acceptable.

Expected length of your commentary: on average, at least half a page per section (Sections 1–6). The overall summary (Section 7) should be at least one page. Brief, superficial comments will receive a low mark for review quality.

Scoring scale:

Each section is scored on a scale of 1 to 5. The descriptors below apply to all sections unless otherwise stated.

Score	1 – Poor	2 – Weak	3 – Adequate	4 – Good	5 – Excellent
	Little or no attempt; major issues throughout.	Partial effort; significant gaps or errors remain.	Meets minimum expectations; some issues present.	Solid work; only minor weaknesses.	Thorough and correct; exceeds expectations.

Submission + Use of AI:

Submit your Review in PDF format, alongside the .zip folder of the code you reviewed. All reports will be automatically checked for AI generation via TurnitIn and GPTZero. Any report that was generated using AI will automatically receive zero marks and will be reported to the IMC course director. This includes "just polishing the language/spell checking".

0Reviewer and Repository Identification

Complete all fields below before proceeding to the review sections. The repository checkout date is important: it establishes the exact snapshot of code you are reviewing. Do not update the repository after you begin writing your report.

Date of repository checkout:	13/04/2026
Repository URL (GitHub link):	https://github.com/IMC-UAS-Krems/assignment-1-GhostDog46
GitHub username of reviewed student:	GhostDog46
Full name of reviewed student:	
Full name of reviewer:	Luka Herceg
Reviewer's student ID:	25/1/0855/070

Section 1 — Completeness – Class Implementation

The assignment requires the implementation of a set of classes distributed across six source files in src/streaming/. In this section you assess how many of these classes have been implemented, which remain as empty stubs, and which are missing entirely. A class is considered “implemented” if its constructor and core methods contain substantive logic — not merely a pass statement or a raise NotImplementedError.

How to proceed:

Open each source file listed below and assess the state of every class. For each class, mark the status as Implemented, Partial (constructor or some methods present but incomplete), or Missing (stub only). Then write a summary commentary below the checklist.

	tracks.py	**Status**
☐	**Track** Abstract base class	Implemented
☐	**Song** Music track associated with an artist	Implemented
☐	**SingleRelease** Song released as a standalone single	Implemented
☐	**AlbumTrack** Song that is part of an album	Implemented
☐	**Podcast** Podcast episode	Implemented
☐	**InterviewEpisode** Interview-format podcast	Implemented
☐	**NarrativeEpisode** Narrative-format podcast	Implemented
☐	**AudiobookTrack** Chapter from an audiobook	Implemented
	users.py	**Status**
☐	**User** Base class for all users	Implemented
☐	**FreeUser** Free-tier user	Implemented
☐	**PremiumUser** Paid subscriber	Implemented
☐	**FamilyAccountUser** Premium user managing a family account	Implemented
☐	**FamilyMember** Profile belonging to a family account	Implemented

	artists.py / albums.py / playlists.py / sessions.py	Status
☐	**Artist** Music artist or content creator	Implemented
☐	**Album** Ordered collection of AlbumTrack objects	Implemented
☐	**Playlist** User-curated ordered collection of tracks	Implemented
☐	**CollaborativePlaylist** Playlist with multiple contributors	Implemented
☐	**ListeningSession** Records a user listening to a track	Implemented

Score	1 – Poor	2 – Weak	3 – Adequate	4 – Good	5 – Excellent
5	Little or no attempt; major issues throughout.	Partial effort; significant gaps or errors remain.	Meets minimum expectations; some issues present.	Solid work; only minor weaknesses.	Thorough and correct; exceeds expectations.

Your score (mark one):	1	2	3	4	5

Commentary: Summarise which classes are implemented, partially implemented, or missing. Note any patterns — e.g., are base classes complete but subclasses missing? Reference specific files and class names.

In tracks.py, the full class hierarchy is implemented, including Song, Podcast, and AudiobookTrack, along with their respective subclasses. These subclasses are designed as data specialisations, and their implementations reflect this appropriately.

In users.py, the base class User includes all expected functionality, such as managing listening sessions and aggregating listening data. The subclasses FreeUser, PremiumUser, FamilyAccountUser, and FamilyMember are all present and correctly structured. They integrate properly with the rest of the system, particularly in the query methods located in platform.py.

The remaining modules are also fully implemented. The Artist and Album classes include the required attributes and methods for managing tracks, while ListeningSession correctly represents user listening activity. In playlists.py, both Playlist and CollaborativePlaylist are implemented as specified. The Playlist class includes a correct implementation of total_duration_seconds(), and CollaborativePlaylist properly manages contributors, including handling edge cases such as duplicate entries and protecting the original creator of the playlist.

Section 2 — Completeness – Query Methods (Q1–Q10)

In addition to the classes, the assignment requires the implementation of 10 analytical query methods in the StreamingPlatform class (file: platform.py). These methods constitute a significant and independently assessable deliverable, as they involve non-trivial logic over the platform's data model. Assess each method separately.

A method is considered Implemented if it contains substantive logic and appears to address the stated specification. Mark it Partial if an attempt has been made but the logic is incomplete or clearly incorrect, and Missing if only a stub remains.

☐	**Q1: total_listening_time_minutes(start, end)** Total cumulative listening time for a time window	Implemented
☐	**Q2: avg_unique_tracks_per_premium_user(days)** Average unique tracks per PremiumUser in last N days	Implemented
☐	**Q3: track_with_most_distinct_listeners()** Track with the highest number of distinct listeners	Implemented
☐	**Q4: avg_session_duration_by_user_type()** Average session duration ranked by user subtype	Implemented
☐	**Q5: total_listening_time_underage_sub_users_minutes(age_threshold)** Listening time for underage FamilyMembers	Implemented
☐	**Q6: top_artists_by_listening_time(n)** Top N artists by cumulative listening time (Songs only)	Implemented
☐	**Q7: user_top_genre(user_id)** Most-listened genre and percentage for a given user	Implemented
☐	**Q8: collaborative_playlists_with_many_artists(threshold)** CollaborativePlaylists exceeding artist threshold	Implemented
☐	**Q9: avg_tracks_per_playlist_type()** Average track count per Playlist vs CollaborativePlaylist	Implemented
☐	**Q10: users_who_completed_albums()** Users who have listened to every track on at least one album	Implemented

Score	1 – Poor	2 – Weak	3 – Adequate	4 – Good	5 – Excellent
5	Little or no attempt; major issues throughout.	Partial effort; significant gaps or errors remain.	Meets minimum expectations; some issues present.	Solid work; only minor weaknesses.	Thorough and correct; exceeds expectations.

Your score (mark one):	1	2	3	4	5

Commentary: For each implemented or partial method, briefly describe what you observed. Does the logic appear to address the specification? Are there obvious algorithmic issues (e.g., incorrect filtering, wrong aggregation, off-by-one in date ranges)? Cite method names and describe what you found.

All ten query methods in platform.py are implemented and contain meaningful logic which is what was asked for us in the assignment.

Each method appears to follow the specification closely. For example, Q1 correctly filters sessions within a time range and aggregates listening time. Q2 calculates the average number of unique tracks per premium user and makes appropriate use of tuples to avoid duplicates. Q3 identifies the track with the most distinct listeners using a suitable approach.

More complex queries are also handled well. Q6 correctly aggregates listening time per artist, and Q7 determines a user's most listened genre along with its percentage. Q8 and Q9 operate on playlists and produce the expected results, while Q10 correctly checks whether users have completed albums by listening to all tracks.

There are some small implementation details that could be improved, such as using isinstance() instead of type(), but these do not affect the overall correctness of the methods.

Overall, all query methods are present, functional, and well implemented.

Section 3 — Correctness and Functionality

This section evaluates whether the implemented code actually behaves correctly — that is, whether it produces the right results for valid inputs. You should assess correctness both by running the provided test suite and by reading the code critically.

Step 1 — Run the test suite.

Follow the setup instructions in the README to create a virtual environment, install dependencies (pip install -e .), and run pytest. Record the outcome below. If the test suite fails to run at all (e.g., import errors), note the error message.

pytest output summary

Total tests collected:	85
Tests passed:	85
Tests failed:	0
Tests with errors:	0
Tests skipped:	0

Step 2 — Code reading.

Beyond test results, read the implementation of selected methods critically. Consider the following guiding questions in your commentary:

- Do the implemented classes and methods handle edge cases? For example: what happens if an empty list is passed, if a user ID does not exist, or if a date range contains no sessions?
- Are there any obvious logical errors — for instance, incorrect use of comparison operators, off-by-one errors, or methods that always return a default value without real computation?
- Do the method signatures and return types match the specification (e.g., returning None where specified, returning the correct collection type)?
- Does the code make correct use of inheritance — for example, calling super().__init__() where appropriate?

Score	1 – Poor	2 – Weak	3 – Adequate	4 – Good	5 – Excellent
4	Little or no attempt; major issues throughout.	Partial effort; significant gaps or errors remain.	Meets minimum expectations; some issues present.	Solid work; only minor weaknesses.	Thorough and correct; exceeds expectations.

Your score (mark one):	**1**	**2**	**3**	**4**	**5**

Commentary: Report your pytest results and describe the key failures or successes. Identify at least two specific correctness observations from reading the code, citing file, class, and method names. If all tests pass, comment on whether the implementation appears robust beyond the provided test cases.

All 85 pytest cases passed successfully with no failures or errors. This proves the implementation works correctly for all tested scenarios and meets the required functionality.

Correctness Observation

1. **Correct handling of main features**
   In platform.py, key methods such as track_with_most_distinct_listeners() and user_top_genre() generally work as expected and produce correct results. These methods also handle common edge cases, which helps explain why all tests pass successfully.
2. **Good handling of missing or empty data**
   The code deals well with situations where there is no data or incomplete information, returning reasonable default values instead of causing errors.

Overall, the implementation is correct and performs well across all test cases. The code is clear and handles most situations properly. There are only minor improvements needed, mainly related to flexibility and scalability.

Section 4 — Test Coverage and Extension

Students were encouraged to extend the provided test suite with additional test cases of their own. This section assesses whether additional tests have been written, whether they are executable, and whether they contribute meaningfully to coverage of the implementation.

Step 1 — Check for additional tests.

Navigate to the tests/ directory and examine the test files. Compare them against the original assignment skeleton (available in the base repository). Look for any test functions or classes that have been added by the student beyond what was provided.

Step 2 — Run the extended test suite.

Run pytest tests/ -v and observe whether all tests — both original and added — execute without errors. A test that raises a syntax error or an import error does not count as an executable test.

Guiding questions for your commentary:

- How many additional test cases (functions or methods) have been added? Provide an approximate count.
- Do the additional tests target meaningful scenarios — for example, edge cases (empty inputs, boundary values, invalid arguments) or specific query methods (Q1–10) that are not well covered by the provided tests?
- Are the test cases well-structured? Do they use clear, descriptive names and contain appropriate assertions?
- Do all tests (original and added) pass, or are some failing? If failing, is this due to a bug in the implementation or a flaw in the test itself?

Note: if no additional tests have been written, score this section accordingly and mention it explicitly in your commentary. Do not penalise the student for an incomplete implementation — only assess whether an honest effort was made to extend the tests.

Score	1 – Poor	2 – Weak	3 – Adequate	4 – Good	5 – Excellent
5	Little or no attempt; major issues throughout.	Partial effort; significant gaps or errors remain.	Meets minimum expectations; some issues present.	Solid work; only minor weaknesses.	Thorough and correct; exceeds expectations.

Your score (mark one):	**1**	**2**	**3**	**4**	**5**

Commentary: Describe what you found in the tests/ directory. How many additional tests were added? Do they run successfully? Do they cover edge cases or scenarios not addressed by the original suite? Has the conftest.py been modified using extended/additional fixtures. Give specific examples of good or missing test cases.

The test_public.py file has been expanded to include all the required tests, with approximately 12 additional test cases added. total_listening_time_underage_sub_users_minutes is a good test using a realistic family setup. It tests two different age thresholds, which helps confirm that the age filtering logic works correctly in different situations. The test also uses a controlled scenario with one listening session, making it easy to understand what should happen. Overall, it effectively verifies both the filtering of underage users and the correct calculation of listening time.

Section 5 — Code Style and Readability

Readable code is code that another developer can understand quickly and maintain with confidence. This section assesses whether the implementation follows consistent conventions and is written in a clear, structured way. You are not expected to apply any specific style guide by name — focus on the general principles below.

Naming conventions.

Are variables, methods, classes, and parameters given names that clearly reflect their purpose? For example, a variable storing a list of listening sessions should not be named x or lst. Class names should start with a capital letter; method and variable names should use lowercase with underscores (e.g., get_user, not getUser).

Function and method length.

Are methods focused on a single task and of reasonable length? A method that spans more than 30–40 lines without clear structure is often a candidate for decomposition into smaller helper methods. Note any particularly long or complex methods.

Nesting and complexity.

Deeply nested conditional logic (e.g., four levels of if/for blocks) is difficult to follow and error-prone. Note any instances where nesting could be reduced through early returns, helper methods, or restructuring.

Consistency.

Is the style applied consistently throughout the code? Inconsistencies — for example, mixing naming conventions, using both single and double quotes without reason, or inconsistent indentation — make a codebase harder to maintain.

Redundancy and dead code.

Are there commented-out blocks of code, unused imports, or duplicate logic that could be removed or consolidated?

Score	1 – Poor	2 – Weak	3 – Adequate	4 – Good	5 – Excellent

4	Little or no attempt; major issues throughout.	Partial effort; significant gaps or errors remain.	Meets minimum expectations; some issues present.	Solid work; only minor weaknesses.	Thorough and correct; exceeds expectations.

Your score (mark one):	**1**	**2**	**3**	**4**	**5**

Commentary: Identify at least three concrete observations about code style or readability, each referencing a specific location (file, class, or method). Balance your observations — note both strengths and areas for improvement.

The code is generally clean, consistent, and easy to read. Naming conventions are applied correctly throughout the project, with classes using PascalCase and methods using snake_case.

Most methods are well structured and focus on a single task, which makes the code easier to follow. The use of Python features such as list comprehensions and generator expressions is appropriate and helps keep the code concise.

Some of the query methods in platform.py are slightly longer and involve multiple nested loops, which can make them harder to read at first. These could be improved by breaking the logic into smaller helper methods.

Section 6 — Documentation and Comments

Documentation is an integral part of professional software development. In Python, the primary form of documentation is the docstring — a string literal placed immediately after the definition of a module, class, or function/method. Good docstrings explain what a piece of code does, what its parameters mean, and what it returns. Inline comments, used sparingly, explain why a non-obvious decision was made.

Class docstrings.

Does each class have a docstring that describes its purpose and any key attributes? For example, the ListeningSession class should document what a session represents and which attributes it holds.

Method docstrings.

Do methods — particularly the 10 query methods — have docstrings that describe their behaviour, parameters, and return values? The skeleton already provides some docstrings; assess whether the student has maintained or extended them.

Inline comments.

Are inline comments used where the logic is non-obvious? A comment above a complex list comprehension or a non-trivial algorithm is helpful. Comments that merely restate what the code already clearly expresses (e.g., # increment counter above count += 1) add noise rather than clarity.

Completeness.

Are there important classes or methods that lack any documentation whatsoever? Note specific omissions, particularly in the query methods, which are the most complex part of the implementation.

Score	1 – Poor	2 – Weak	3 – Adequate	4 – Good	5 – Excellent
	Little or no attempt; major issues throughout.	Partial effort; significant gaps or errors remain.	Meets minimum expectations; some issues present.	Solid work; only minor weaknesses.	Thorough and correct; exceeds expectations.

Your score (mark one):	1	2	3	4	5

Commentary: Assess the overall quality of documentation. Are docstrings present, accurate, and informative? Are inline comments used appropriately? Cite at least two specific examples — one positive and one that could be improved.

The code includes basic documentation which was provided withing the base repository, such as module-level docstrings and some class descriptions.

However, many methods—especially the query methods in platform.py—lack detailed docstrings explaining their parameters, return values, and behaviour. Given that these methods contain the most complex logic in the project, more thorough documentation would be useful.

Inline comments are used sparingly, but there are some areas where additional explanation would improve clarity, particularly in more complex methods.

Section 7 — Overall Summary and Final Score

Having completed the six evaluation sections above, provide an integrative summary of your overall impression of the submission. This section should synthesise your observations into a coherent narrative — it is not a repetition of individual section scores, but a holistic assessment of the work as a whole.

Expected length: at least one full page.

1. Overall impression (2–4 sentences)

Describe the general state of the submission. Is the codebase at an early stage, a mid-stage prototype, or largely complete? Is the overall approach sound?

Write your overall impression here.

The codebase is largely complete, with the main area for improvement being the addition of docstrings and comments to clarify some of the more complex sections. Overall, the implementation is well-structured and follows a clear, logical approach, making the code easy to read and understand.

2. Top strengths (identify at least 2, ideally 3)

What does the submission do particularly well? Reference specific classes, methods, or test cases. Be precise — "the code is good" is not a strength; "all base class constructors are complete and correctly use super().__init__()" is.

Strength 1:	Strong and complete query implementation

Strength 2:	**Clean and readable code structure**

Strength 3:	

3. Priority areas for improvement (identify at least 2, ideally 3)

What are the most important things the student should address before the final submission? Order these by priority — the first item should be the most critical. Each improvement area should reference a specific location or pattern in the code.

Priority 1:	Lack of detailed documentation

Priority 2:	Inconsistent use of type checking (type() vs isinstance())

Priority 3:	

4. Extended commentary

Expand on your observations in free prose. You may discuss how the different dimensions of the submission relate to one another — for example, whether incomplete class implementations affect the query methods, or whether good documentation partially compensates for missing functionality. You may also note anything not captured in the earlier sections.

Extended commentary:

Overall, this is a well-structured submission that mostly follows the UML and works as expected. From the codebase itself, it is easily seeable that the coder is quite comfortable with Python and the use of classes and methods, alongside that they know how to expand their test case to cover more obscure cases. Most of the classes are implemented correctly, especially in tracks.py, and the system is generally consistent and easy to follow. The users.py module is also mostly solid, although the family account parts are not as fully developed and could be improved. The other main parts of the system, like Artist, Album, ListeningSession, and playlists, are well implemented and handle normal cases and edge cases properly. The query methods in platform.py are a strong point, with all ten working correctly and using good logic, even though there are a few small style issues like using type() instead of isinstance(). Overall functionality is correct, and the tests all pass. The code is generally clean and readable, although a few longer methods could be broken up to make them easier to follow. There is some basic documentation, but more detail—especially for the more complex methods—would help a lot. Overall, it is a strong piece of work with good implementation, with only small improvements needed in family accounts, testing, and documentation.

Final Overall Score

Based on your assessment across all six sections, assign a single overall score. This should reflect a holistic judgement — not merely the arithmetic mean of the individual section scores.

Overall score (mark one):	**1**	**2**	**3**	**4**	**5**